MONITOR Z · L00.3

Clase 3 — Por qué no alcanza con «preguntar cada tanto»

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L00.3 — Clase 3 · Por qué no alcanza con «preguntar cada tanto»

Temario de la L00: 1. El demonio en Unix ✅ · 2. La misma idea en z/OS ✅ · **3. Por qué no alcanza con «preguntar cada tanto»** · 4. Las piezas del monitor, y dónde estamos hoy.

Objetivo de esta clase: medir, con los datos de JOB34879, las dos cosas que se pagan al **preguntar cada N segundos**: **llegar tarde** (latencia) y **preguntar en vano**. Con eso se entiende por qué el diseño tiene tres niveles y por qué el último es por eventos.

> 📚 Fuentes: - IDEA-005 en `IDEAS_cierres_produccion.md` (2026-09-30): los costos de SDSF y FTP-JES, y los tres niveles. - `monitor_z/asm/ZTKMON.asm` (v0.9): `+EVT INI` lleva la hora en que **el monitor lo vio** (`TSAHORA`, línea 527) y `+EVT FIN` lleva `TI=`, el inicio que dice **JES** (`STVBXTST`, líneas 739 y 829). - El SYSPRINT de **JOB34879** (DESA, 2026-10-22): los 12 `+EVT`, las 6 `+DIAG COSTO` y el `+FIN ZTKMON … EVENTOS=00012`. - *z/OS MVS Programming: Authorized Assembler Services Reference* (`ENFREQ`) y el mapeo `IAZENF70`: el nivel C.

1. Preguntar cada tanto (*polling*)

Analogía: el chico en el asiento de atrás que pregunta **«¿ya llegamos?»** cada cinco minutos. Gasta saliva aunque no haya novedad, y cuando por fin llegan se entera hasta la próxima pregunta. Lo ideal es que el chofer **avise** al llegar.

Forma	Quién trabaja	Cuándo te enterás	Lo que se gasta
🔁 **Preguntar cada N s** (*polling*)	el que pregunta, siempre	en la próxima pregunta: hasta **N s tarde**	cada pregunta, haya o no novedad
🔔 **Que te avisen** (eventos)	el que sabe, sólo cuando pasa algo	**enseguida**	nada mientras no pase nada

ZTKMON hoy **pregunta**: cada 5 s le pide a JES la lista de los que ejecutan y la de los que terminaron (SSI 80). Duerme bien entre una pregunta y otra (la clase 1), pero sigue siendo *polling*.

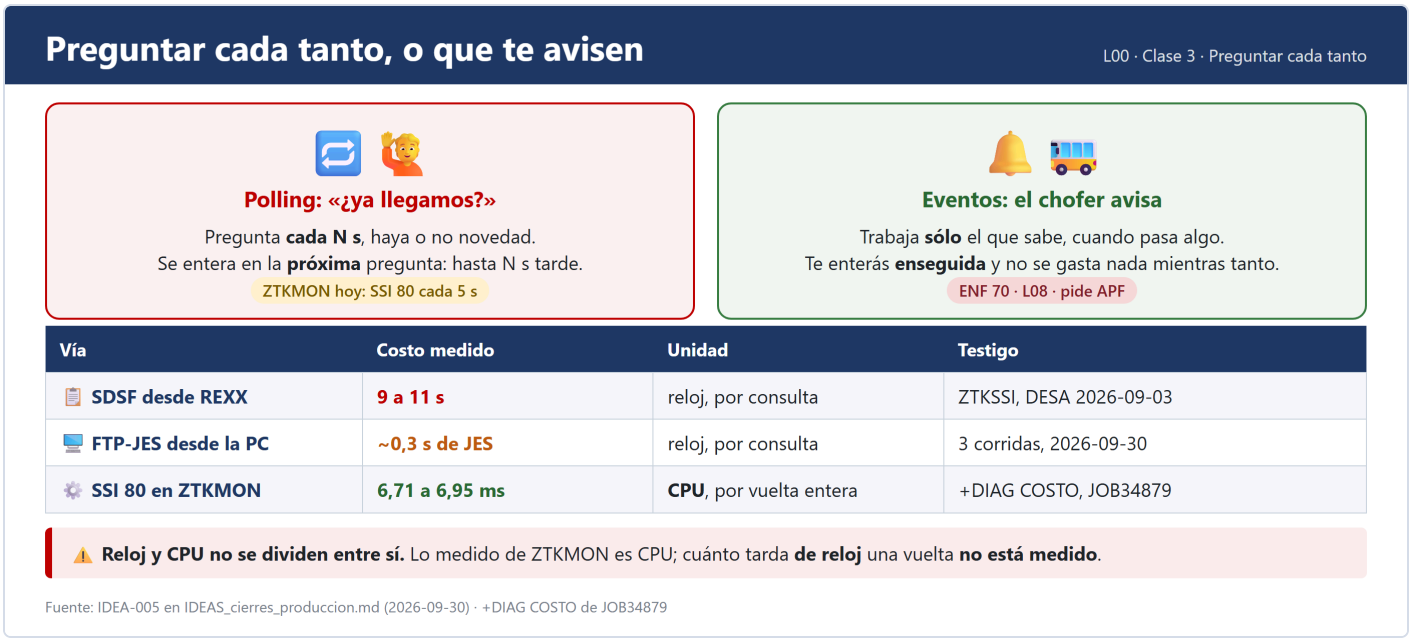


Lámina L00-17_polling_o_aviso

2. Cuánto cuesta cada pregunta, según cómo se haga

Vía	Costo medido	Unidad	Evidencia
SDSF desde REXX (`ISFEXEC ST`)	**9 a 11 s**	tiempo de reloj por consulta: recorre todo el spool	cabecera de `PPCE.ZTK.ASM(ZTKSSI)`, DESA 2026-09-03
FTP-JES desde la PC (zowe)	**~0,3 s** de JES (3,42–3,46 s en total, 3,16 s son el arranque de la CLI)	tiempo de reloj	3 corridas, 144 filas, 2026-09-30
SSI 80 desde ZTKMON	**6,71 a 6,95 ms** por vuelta	**CPU**, no reloj, y de la vuelta entera (dos consultas a JES y el resto)	`+DIAG COSTO`, JOB34879

Ojo con las unidades. Las dos primeras son **tiempo de reloj**; la tercera es **CPU**. No se pueden dividir una por la otra. Lo que sí dice la tabla: la SSI 80 cuesta **menos de 7 ms de CPU** por vuelta, y eso es lo que pagamos 360 veces en 30 minutos. Cuánto tarda **de reloj** una vuelta **no está medido**.

La conclusión de IDEA-005 sigue en pie: el costo lo pone **la interfaz** de JES (SDSF contra SSI) y **lanzar un job por consulta**, no el lenguaje.

3. El primer precio: llegar tarde

ZTKMON escribe dos horas en sus eventos: en el `+EVT INI`, `T=` es la hora en que **él lo vio**; en el `+EVT FIN`, `TI=` es la hora de inicio según **JES**. La resta es cuánto tardó en enterarse. JOB34879, los 6 ZTKDOK:

Job	Inició según JES (`TI=`)	ZTKMON lo vio (`+EVT INI T=`)	Tarde
JOB34883	11:40:28	11:40:33	**5 s**
JOB34884	11:40:58	11:41:03	**5 s**
JOB34887	11:42:00	11:42:03	**3 s**
JOB34888	11:42:30	11:42:36	**6 s**
JOB34891	11:44:06	11:44:09	**3 s**
JOB34892	11:44:36	11:44:39	**3 s**

De 3 a 6 segundos, con un intervalo de 5 s. Encaja: un job que arranca justo después de una vuelta espera casi toda la siguiente. El 6 s de JOB34888 se explica porque las dos horas están truncadas al segundo (una diferencia real de 5,x s se ve como 6). Es una explicación, no una medición.

Con el nivel A (cada 60 s desde la PC) sería mucho peor. Cada ZTKDOK duró unos 30 s: JOB34883 arrancó a las 11:40:28 y su fin se vio a las 11:40:58. Un job de 30 s que cae entre dos consultas separadas por 60 s **nunca aparece como «ejecutando»**: la PC lo ve directamente terminado, o no lo ve si ya se purgó. Es un cálculo sobre esos datos, no una prueba con el nivel A.

El primer precio: llegar 3 a 6 segundos tarde

L00 · Clase 3 · Preguntar cada tanto

Job	🏁 Inició según JES (TI= del FIN)	👀 ZTKMON lo vio (T= del INI)	🐢 Tarde
JOB34883	11:40:28	11:40:33	**5 s**
JOB34884	11:40:58	11:41:03	**5 s**
JOB34887	11:42:00	11:42:03	**3 s**
JOB34888	11:42:30	11:42:36	**6 s**
JOB34891	11:44:06	11:44:09	**3 s**
JOB34892	11:44:36	11:44:39	**3 s**

De dónde sale cada hora: el `T=` del INI es `TSAHORA`, cuando **el monitor** lo vio (línea 527). El `TI=` del FIN es `STVBXTST`, el inicio según **JES** (líneas 739 y 829).

🤔 **¿Y el 6 s?**

Con 5 s de intervalo, lo peor serían ~5 s. Las dos horas están **truncadas al segundo**: 5,x s puede verse como 6. *Explicación, no medición.*

Fuente: SYSPRINT de JOB34879 (los 12 +EVT) · ZTKMON.asm v0.9 (líneas 527, 739, 816, 829)

Lámina L00-18_llegar_tarde

4. El segundo precio: preguntar en vano

Dato	Valor	Testigo
Vueltas	**360**	`+DIAG COSTO VUELTAS=00360`
Eventos	**12** (6 INI y 6 FIN)	`+FIN ZTKMON … EVENTOS=00012`
Vueltas con algo para contar	**12**: cada evento tiene una hora distinta	los 12 `T=` del SYSPRINT
Vueltas **sin novedad**	**348 de 360 = 96,7 %**	cuenta
Cuándo hubo eventos	sólo entre las 11:40 y las 11:45	los `T=`

Es decir: de los 2,55 s de CPU, casi todo se gastó en preguntar «¿hay algo nuevo?» y escuchar «no». En DESA con 6 jobs de prueba es poco. En una noche de cierre, con miles de jobs, la pregunta trae listas más largas **cada** vuelta (la deuda **H3** de la R02), aunque la mayoría no haya cambiado.



Lámina L00-19_preguntar_en_vano

5. Los tres niveles del diseño

Nivel	Cómo	Llega tarde	Pregunta en vano	Pide	Estado
A. Polling desde la PC	FTP-JES cada 60 s	hasta **60 s**; los jobs cortos ni se ven	sí, y en la red	nada en el host	descartado como principal
B. Un programa que duerme y pregunta	`SELECT` 5 s + SSI 80	**3 a 6 s** (medido)	**96,7 %** de las vueltas (medido)	un iniciador (o un STC)	✅ **ZTKMON v0.9**
C. Por eventos	listen exit de **ENF 70** que hace `POST` de un ECB; la tarea espera en `WAIT`	«enseguida»	**no**	**APF** + STC	⏳ L08, **no medido**

Por qué B primero y C al final:

1. **B ya sirve:** 3 a 6 s alcanzan para un cierre donde los jobs duran minutos.

2. **B enseña lo que dice JES:** las fases, los CC y los tiempos que C también va a necesitar.
3. **C reusa casi todo B:** cambia **quién despierta** al bucle. Hoy lo despierta el plazo del `SELECT`; con ENF 70 lo despierta el `POST` que hace el exit cuando JES avisa. Es el semáforo de SOPES2 de la clase 1.
4. **C pide permisos** (APF y STC) que dependen de Sistemas.

> Lo que dice IDEA-005 sobre ENF 70 (SELECT, DESELECT con NRM/CC/JCL/CAN/ABN/FLU, CHANGE y PURGE) sale del mapeo `IAZENF70` y del manual. **No se probó.** Que C llegue «enseguida» es lo que promete el diseño, no un número.

Los tres niveles, con lo que ya se midió

L00 · Clase 3 · Preguntar cada tanto

Nivel	Llega tarde	Pregunta en vano	Pide	Estado
A · PC cada 60 s	**hasta 60 s; un job de 30 s puede no verse nunca ejecutando**	sí, y por la red	nada en el host	descartado como principal
B · duerme y pregunta	**3 a 6 s (medido)**	**96,7 % (medido)**	un iniciador, o un STC	✅ **ZTKMON v0.9**
C · eventos (ENF 70)	**«enseguida» (promesa del diseño)**	**no**	**APF + STC**	⏳ L08 · no medido

🧱 Por qué B primero
Ya sirve (3–6 s alcanzan si los jobs duran minutos), enseña lo que dice JES y no pide permisos.

🔄 Qué cambia en C
Quién despierta al bucle. Hoy, el plazo del `SELECT`. En C, el `POST` que hace el exit de ENF 70 en un ECB: el semáforo de SOPES2.

El nivel A
Cada ZTKDOK duró ~30 s (inició 11:40:28, su fin se vio 11:40:58). Con 60 s entre consultas cae **entre dos**. *Cálculo, no prueba.*

Fuente: IDEA-005 (tres niveles) · SYSPRINT de JOB34879 · L08 (ENF 70) · z/OS MVS Programming: Authorized Assembler Services Reference (ENFREQ)

Lámina L00-20_tres_niveles

6. Qué cambia para el monitor

Hoy (B)	Con C	Qué lo trae
se entera 3 a 6 s tarde	se entera cuando JES avisa	L08 + APF
96,7 % de las vueltas sin novedad	despierta sólo si hay algo	L08
la lista de JES se pide entera cada vuelta (H3)	llega un evento por job	L08; antes, medir H3
un solo motivo para despertar: el plazo o el puente	un motivo más: el ECB del exit	`WAIT ECBLIST` o el `SELECTEX` de EZASOKET

Resumen

Idea	En una línea	Testigo
Polling	preguntar cada N s, haya o no novedad	ZTKMON: SSI 80 cada 5 s
Costo por pregunta	SDSF 9–11 s de reloj, FTP-JES ~0,3 s de reloj, SSI 80 < 7 ms de CPU por vuelta	IDEA-005 · `+DIAG COSTO`
Llegar tarde	3 a 6 s con un intervalo de 5	`T=` del INI contra `TI=` del FIN, 6 ZTKDOK
Preguntar en vano	348 de 360 vueltas sin novedad	JOB34879
Nivel A	con 60 s, un job de 30 s puede no verse nunca ejecutando	cálculo con los ZTKDOK
Nivel C	que JES avise: ENF 70, `POST`, `WAIT`	no medido; pide APF y STC

Preguntas de comprobación

1. Si cambiás el PARM a `'ZTKD* 1 30 8010'` (una vuelta por segundo), ¿qué esperás que pase con la tardanza de 3 a 6 s y

con los 2,55 s de CPU? ¿Qué pierde y qué gana el monitor?

1. ¿Por qué el nivel C no tiene «vueltas sin novedad», y qué parte del bucle de ZTKMON cambia para pasar de B a C?

Mis respuestas

Relacionado: *L00.2 - Clase 2 - La misma idea en z-OS - STC, JES y address spaces* · *L03 - Hablarle a JES por SSI 80* · *L08 - Eventos de verdad - ENF 70* · *R02 - Mapa de las mejoras A1-A12 contra ZTKMON v0.9* · siguiente: L00.4.

Notas